Minimal BLE Stack Walkthrough

Architecture, Code Flow, and Peripheral Packet Behavior

2026-03-27

Purpose

This document explains how the BLE stack in this repository works today. It is written against the current code in the repository stack and bundled example:

- `stack/include/nrf_ble.h`
- `stack/core/ble_stack.c`
- `stack/core/ble_internal.h`
- `stack/core/ble_runtime.c`
- `stack/core/ble_controller.c`
- `stack/gatt/ble_gatt_server.c`
- `stack/radio/radio_driver.h`
- `stack/radio/radio_driver.c`
- `examples/custom_ble_stack_demo/main.c`

The goal is not to teach the whole BLE specification. The goal is to help you read this stack comfortably and understand how data, packets, timers, and callbacks move through the code.

Design Summary

This stack is intentionally small. It implements only the parts needed for a minimal BLE peripheral.

At a high level:

1. `ble_stack.c` exposes the public API.
2. `ble_runtime.c` owns shared state, small utilities, and deferred event delivery.
3. `ble_controller.c` owns the BLE link-layer flow and packet scheduling.
4. `radio_driver.c` owns direct access to the nRF radio peripheral.
5. `ble_gatt_server.c` owns the ATT/GATT database and request processing.
6. `examples/custom_ble_stack_demo/main.c` shows how an application configures services and reacts to events.

Key design choices:

- Advertising and GATT are configured separately.
- Services and characteristics come from user-provided tables, not hardcoded stack entries.
- BLE stack events and GATT callbacks are deferred to a low-priority software interrupt.
- ATT MTU stays fixed at 23.

- There is no segmentation or reassembly.
- Notifications use a single pending TX slot.

File Map

Public Layer

- `stack/include/nrf_ble.h` Public types, constants, callback types, and API entry points.
- `stack/core/ble_stack.c` Thin wrapper that initializes shared state and forwards work to the controller and ATT/GATT layers.

Internal Core

- `stack/core/ble_internal.h` Internal constants, shared structs, internal function prototypes, and the globally shared internal state declarations.
- `stack/core/ble_runtime.c` Shared state definitions, queue helpers, utility functions, deferred event delivery, and identity address loading.
- `stack/core/ble_controller.c` BLE controller flow: advertising, connection acceptance, connection-event timing, data-channel hopping, LL control handling, and ATT packet transport.

Radio Layer

- `stack/radio/radio_driver.h` Small radio abstraction used by the controller for packet config, timing, event waits, tasks, and address setup.
- `stack/radio/radio_driver.c` Direct `NRF_RADIO` access, interrupt hookup, and the low-level helper implementation behind the controller.

GATT Layer

- `stack/gatt/ble_gatt_server.c` Builds the local ATT database, processes ATT requests, tracks CCCD state, and generates notifications.

Example App

- `examples/custom_ble_stack_demo/main.c` This bundled demo application creates the custom service table, registers callbacks, starts advertising, and periodically sends a notification.

Runtime State Containers

One of the recent cleanups in this codebase was grouping scattered globals into relevant structs. The important shared state now looks like this.

`m_host`

`m_host` stores host-side configuration:

- advertising name
- advertising flags
- TX power

- advertising interval
- included service UUID
- optional service data
- stored GAP preferred connection parameters

This is mostly configuration, not packet-exchange runtime state.

`m_controller`

`m_controller` stores a small amount of top-level controller state:

- current advertising-channel index
- whether advertising is active
- whether the app timer for advertising has already been created

`m_link`

`m_link` stores connection-specific state:

- connected flag
- connection interval
- supervision timeout
- hop increment and channel map
- access address and CRC init
- sequence numbers
- event counter
- pending channel-map update information

This struct is the logical "connected link" state.

`m_evt_handler`

`m_evt_handler` stores the registered BLE stack event callback pointer.

This one handler receives the stack-level GAP and GATT events:

- `BLE_GAP_EVT_CONNECTED`
- `BLE_GAP_EVT_DISCONNECTED`
- `BLE_GAP_EVT_SUPERVISION_TIMEOUT`
- `BLE_GAP_EVT_CONN_UPDATE_IND`
- `BLE_GAP_EVT_TERMINATE_IND`
- `BLE_GATT_EVT_MTU_EXCHANGE`

GATT callbacks are still not stored here because each characteristic owns its own GATT event handler.

`m_ctrl_rt`

`m_ctrl_rt` stores controller runtime buffers and transient radio state:

- advertising PDU buffer
- connection RX buffer
- connection TX buffer

- last transmitted data PDU
- pending data PDU to send on the next event
- advertiser identity address
- advertiser address type bit
- flags for retransmission and pending TX
- connection timer initialization state
- next scheduled connection event tick

This is the most important "packet movement" struct.

`m_diag`

`m_diag` stores lightweight debug state:

- whether first data packet was seen
- ring buffer of LL control opcodes
- ring buffer of ATT opcodes
- packet trace ring buffer
- packet trace budget

These are exposed through `ble_diag_take_*()` helpers.

Public API and Its Meaning

The application-facing API is intentionally small.

`ble_stack_init(void)`

This resets the internal runtime state, loads a static identity address, initializes deferred event dispatch, and brings up the radio controller runtime.

This function does not start advertising by itself.

`ble_register_evt_handler(ble_evt_handler_t handler)`

This installs the unified BLE stack event callback.

The callback receives a flat `ble_evt_t` with:

- `evt_type`
- `conn_interval_ms`
- `supervision_timeout_ms`
- `requested_mtu`
- `response_mtu`
- `effective_mtu`

Only the MTU fields are meaningful for `BLE_GATT_EVT_MTU_EXCHANGE`.

`ble_adv_init(const ble_adv_config_t *p_config)`

This stores advertising configuration in `m_host`:

- device name

- advertising flags
- TX power
- advertising interval
- included service UUID
- optional service data

It does not start advertising.

Important detail:

- The advertising name is also reused as the GAP device name during `ble_gatt_server_init()`.

`ble_gatt_server_init(ble_gatt_service_t *p_services, uint8_t service_count)`

This rebuilds the ATT/GATT database from scratch:

- fixed GAP/GATT attributes are added first
- custom services and characteristics are appended after that
- handles are written back into the user's service and characteristic structs

`ble_gap_init(const ble_gap_conn_params_t *p_params)`

This stores preferred connection parameters in `m_host`.

Current limitation:

- these values are stored but not yet actively used for LL connection parameter update and not exposed through a fixed PPCP attribute

So today this API is more of a placeholder for future extension than an active protocol feature.

`ble_start_advertising()`

Starts periodic advertising.

`ble_stop_advertising()`

Stops the advertising timer and disables the radio.

`ble_notify_characteristic(...)`

Builds an ATT notification and asks the controller to queue it for the next connection event.

This succeeds only if:

- there is an active connection
- notifications are enabled for that characteristic
- the single pending TX slot is free

Example App Flow

The example in `examples/custom_ble_stack_demo/main.c` is the easiest way to see intended usage.

Its startup sequence is:

1. start clocks
2. initialize debug logging
3. initialize LEDs
4. call `ble_stack_init()`
5. call `ble_register_evt_handler(ble_evt_handler)`
6. call `ble_adv_init(&m_adv_config)`
7. call `ble_gap_init(&m_gap_conn_params)`
8. call `ble_gatt_server_init(m_custom_services, service_count)`
9. create an application timer for periodic measurement updates
10. start advertising

Notably, the main loop does not poll BLE events.

The loop only runs:

- `debug_log_process()`

This is possible because BLE stack events and GATT events are callback-driven now.

Initialization Flow in Detail

Step 1: `ble_stack_init()`

`ble_stack_init()` performs a clean reset of the shared state:

- clears `m_host`
- clears `m_controller`
- clears `m_link`
- clears `m_ctrl_rt`
- clears `m_evt_handler`
- clears `m_diag`
- restores default advertising flags and interval
- restores default stored GAP connection parameters
- preserves whether the advertising timer object was already created

Then it calls:

1. `controller_load_identity_address()`
2. `ble_evt_dispatch_init()`
3. `controller_runtime_init()`

Step 2: identity address generation

`controller_load_identity_address()` reads device address and device ID from FICR, mixes them with a salt, and creates a static random address.

The top two bits of the last byte are forced to `11`, which makes it a static random address.

Step 3: deferred event dispatcher setup

`ble_evt_dispatch_init()` clears the internal deferred-event queue and enables `SWI1_EGU1` at low priority.

This interrupt is later used to deliver BLE stack events and GATT callbacks outside the time-sensitive radio path.

Step 4: controller runtime setup

`controller_runtime_init()`:

- initializes `app_timer`
- creates the repeated advertising timer if needed
- powers the radio through `radio_driver`
- configures radio ramp-up/default-TX behavior through `radio_driver`
- configures radio timing (`TIFS = 150 us`) through `radio_driver`
- initializes the connection timer machinery
- prepares the radio in advertising format
- registers `radio_evt_handler()` as the radio callback
- enables radio interrupts

Advertising Flow

Advertising is a periodic timer-driven process.

Configuration stage

`ble_adv_init()` copies the advertising settings into `m_host`.

This includes:

- local name
- flags AD field
- TX power AD field
- advertising interval
- included service UUID AD field
- optional service data AD field

Start stage

`ble_start_advertising()` checks:

- not already connected

Then it:

- marks `m_controller.advertising = true`
- prepares the radio for advertising format
- starts the repeated `app_timer`

Timer stage

The repeated app timer calls `adv_timer_handler()`, which calls `ble_advertise()`.

`ble_advertise()`:

1. builds the advertising PDU in `m_ctrl_rt.air_pdu`
2. loops over channels 37, 38, and 39
3. transmits on each channel
4. leaves RX open briefly after each transmission to catch requests

Important detail:

- one advertising event covers all three advertising channels

Advertising packet construction

`host_build_adv_pdu()` fills `m_ctrl_rt.air_pdu` with:

- `ADV_IND` header
- advertiser address
- flags AD structure
- complete local name AD structure, if present
- TX power AD structure
- complete list of 16-bit service UUIDs, if configured
- service data AD structure, if configured

There is no separate scan response payload in the current implementation.

Packet Formats Used by This Stack

This section describes the actual packet shapes the current peripheral stack builds and parses.

Advertising-channel packets

Advertising packets use `ble_ll_adv_pdu_t`:

- 2-byte advertising header
- 6-byte advertiser address
- variable payload made of AD structures

The transmitted advertising PDU is always `ADV_IND`.

The payload is assembled in this order:

1. flags
2. complete local name, if configured
3. TX power
4. complete list of 16-bit service UUIDs, if configured
5. service data, if configured

The total advertising data stays within the normal 31-byte limit.

Connect request packet

The connection-opening packet is `LL_CONNECT_REQ`, represented in code by `ble_connect_req_pdu_t`.

The stack uses these fields from that packet:

- access address
- CRC init
- transmit-window offset
- connection interval
- supervision timeout
- channel map
- hop increment

There is no separate "connect response" packet in this path. Once the connect request is accepted, the controller switches from advertising format to connected format and schedules the first data-channel event.

Connected data-channel packets

Connected packets use `ble_ll_data_raw_pdu_t`:

- 1-byte header
- 1-byte payload length
- variable payload

The connected header carries:

- `LLID` in bits 0-1
- `NESN` in bit 2
- `SN` in bit 3

The stack uses these LLID meanings:

- `01`: empty or continuation data PDU
- `10`: start of L2CAP payload
- `11`: LL control PDU

ATT inside L2CAP inside the LL data PDU

When LLID is `10`, the payload begins with the minimal 4-byte L2CAP header:

- 2-byte L2CAP payload length
- 2-byte CID

For ATT traffic the CID is always `0x0004`.

So an ATT packet on air in this stack looks like:

1. connected LL header
2. connected LL payload length
3. 2-byte L2CAP length
4. 2-byte L2CAP CID `0x0004`
5. ATT opcode and ATT parameters

Because ATT MTU is fixed at 23, the current stack keeps ATT traffic small enough that it fits in one unfragmented L2CAP packet.

How a Connection Is Established

In this peripheral-only stack, a connection starts when a received advertising-channel packet is recognized as `LL_CONNECT_REQ`.

Radio receive path

While not connected, `radio_evt_handler()` uses `m_ctrl_rt.air_pdu` as the active packet buffer.

The handler checks whether the received PDU type is `LL_CONNECT_REQ`. If yes, it calls `controller_apply_connect_request()`.

`controller_apply_connect_request()`

This function parses the connection request and populates `m_link` with:

- connection interval
- supervision timeout
- hop increment
- CRC init
- access address
- channel map

It also:

- resets sequence numbers
- resets retransmission state
- resets diagnostics
- resets ATT/GATT connection state
- stops the advertising timer
- switches the radio into connection format
- starts the connection timer
- queues a deferred `BLE_GAP_EVT_CONNECTED`

First event scheduling

The first connected listening window is scheduled using the transmit window offset from the connection request.

If the peer requested zero offset, the stack uses a small fixed guard delay instead of waiting a full interval.

Connected Link Operation

Once connected, the main behavior is driven by `TIMER2_IRQHandler()` and `radio_handle_connected_packet()`.

Connection timer responsibility

`TIMER2_IRQHandler()` is responsible for connection-event pacing.

On each compare event it:

1. checks that the link is still connected
2. advances the next compare using the connection interval
3. applies supervision timeout logic
4. clears per-interval RX and anchor flags
5. applies pending channel-map updates if their instant is reached
6. starts the next connection event
7. increments the event counter

Starting a connection event

`controller_start_connection_event()`:

- hops to the next data channel
- configures the radio for RX on that channel
- points `PACKETPTR` at `m_ctrl_rt.conn_rx_pdu`
- starts listening

What happens when a data-channel packet arrives

`radio_evt_handler()` sees that `m_link.connected` is true and forwards to `radio_handle_connected_packet()`.

That function does several jobs:

1. copies the received packet from `m_ctrl_rt.conn_rx_pdu`
2. determines whether it is new or a retransmission using `SN`
3. captures the real event anchor from the first valid packet of the interval
4. updates the supervision bookkeeping
5. decides whether the previous TX PDU was acknowledged using `NESN`
6. sends a response immediately to meet BLE turn-around timing
7. only after responding, parses the newly received payload
8. prepares any control or ATT response for the next event

This last point is important.

Current design choice:

- the stack responds immediately with either a retransmission, a queued pending PDU, or an empty PDU
- control and ATT processing from the just-received packet usually prepares a pending PDU for the next connection event, not the same one

This keeps the radio timing simple.

Retransmission model

The stack stores:

- the last transmitted PDU

- whether it is still unacknowledged

If the peer does not acknowledge it, the next response retransmits that same PDU. If the peer acknowledges it, `tx_sn` is toggled and the pending slot can be used for the next transmission.

LL Control Handling

Control PDUs are handled by `ll_control_process()`.

Currently supported behavior includes:

- Channel Map Indication: parsed and applied later at the specified instant
- Feature Request / Slave Feature Request: replies with zeroed feature bits
- Version Indication: replies with a version packet
- Length Request: replies with 27-byte payload support
- PHY Request: replies with 1M PHY
- Terminate Indication: disconnects immediately
- Connection Update Indication: only logged, not actively applied yet
- Unknown opcodes: replied to with `UNKNOWN_RSP`

This is enough for basic interoperability, but still intentionally minimal.

ATT and GATT Database Model

The ATT/GATT layer is simple and table-driven.

Fixed attributes

The stack always builds these fixed handles first:

Handle	Meaning
`0x0001`	GAP service
`0x0002`	Device Name declaration
`0x0003`	Device Name value
`0x0004`	Appearance declaration
`0x0005`	Appearance value
`0x0006`	GATT service

The first custom attribute starts at `0x0007`.

Custom service layout

Each user service contributes:

1. one primary service attribute
2. for each characteristic:
    - one characteristic declaration
    - one characteristic value
    - one CCCD, only if notify is enabled for that characteristic

So a notifying characteristic uses three handles, while a non-notifying characteristic uses two.

Handle write-back

During `ble_gatt_server_init()`:

- each service gets `service_handle`
- each characteristic gets `value_handle`
- notifying characteristics also get `cccd_handle`

These handles are stored back into the user-provided structs so the application can later notify by referring to the same characteristic object.

Device name source

The GAP device name attribute is populated from the advertising name stored in `m_host.adv_name`. If no name was configured, it falls back to `"nrf52-ble"`.

ATT Request Processing

`ble_gatt_server_process_request()` is the central ATT dispatcher.

Supported request types:

- Exchange MTU Request
- Find Information Request
- Find By Type Value Request
- Read By Group Type Request
- Read By Type Request
- Read Request
- Write Request
- Write Command

Unsupported requests return an ATT error response.

MTU behavior

The stack accepts MTU exchange but keeps the effective ATT MTU at 23.

Behavior today:

- peer can send `Exchange MTU Request`
- stack queues `BLE_GATT_EVT_MTU_EXCHANGE` for the application
- stack replies with MTU 23
- internal effective MTU is clamped to 23

This matches the current transport design because there is no fragmentation or reassembly support.

Reads

Reads are served directly from the value buffers associated with attributes in the GATT database.

For custom characteristics, that means the application-owned `p_value` and `p_value_len` are the real backing storage.

Writes

`att_handle_write_req()` writes directly into the attribute value buffer, but it does not silently truncate oversized writes.

Two different behaviors matter:

- characteristic value write
- CCCD write

Characteristic value write

If the target is a characteristic value handle:

- a write larger than the declared `max_len` is rejected
- otherwise the user buffer is updated
- `p_value_len` is updated
- a deferred GATT write event is queued

CCCD write

If the target is a CCCD handle:

- the internal CCCD bytes are updated
- `notifications_enabled` is recomputed
- a deferred GATT event is queued with either `BLE_GATT_EVT_NOTIFY_ENABLED` or `BLE_GATT_EVT_NOTIFY_DISABLED`

Write Request versus Write Command

The stack treats the two write styles differently:

- `WRITE_REQ` returns an ATT response
- `WRITE_CMD` returns no ATT response

If an oversized `WRITE_CMD` arrives, the stack ignores it rather than partially writing the value.

Notification Flow

Notifications start in application code.

Example from `examples/custom_ble_stack_demo/main.c`:

1. timer handler increments the counter value
2. app calls `ble_notify_characteristic(&m_custom_characteristics[0])`

Then the stack does:

1. `ble_notify_characteristic()`
2. `ble_gatt_server_build_notification()`
3. `controller_queue_att_payload()`

What `ble_gatt_server_build_notification()` does

It verifies:

- characteristic runtime exists
- notifications are enabled

Then it builds a small ATT packet:

- opcode `HANDLE_VALUE_NOTIFICATION`
- value handle
- characteristic value bytes

What `controller_queue_att_payload()` does

It wraps the ATT payload in a minimal L2CAP header and stores it in the single pending connection TX slot.

Important limitation:

- there is only one pending ATT TX slot
- if that slot is already occupied, the function returns `false`

That is why notification send attempts can legitimately fail even while connected. The application should tolerate this and try again later if needed.

Deferred BLE and GATT Callback Delivery

The stack no longer expects the main loop to poll BLE events.

Instead, BLE stack events and GATT events are deferred through a software interrupt queue.

Queueing path

When the stack wants to emit an event, it calls one of:

- `ble_evt_notify_gap()`
- `ble_evt_notify_gatt_mtu_exchange()`
- `ble_evt_notify_gatt_characteristic()`

Those functions:

1. create a small event object
2. push it into the internal ring buffer in a critical section
3. pend `SWI1_EGU1`

Dispatch path

`SWI1_EGU1_IRQHandler()` drains the queue and calls:

- the registered BLE event handler for GAP and GATT stack events
- the per-characteristic event handler for GATT characteristic events

This means:

- no BLE event polling is needed in the main loop
- the radio path stays shorter and simpler
- callbacks still do not run in thread context

Important callback context rule:

- BLE and GATT callbacks run in low-priority IRQ context, not in normal main loop context

So callback code should stay short and non-blocking.

Packet-By-Packet Peripheral Flow

This section focuses on what actually goes over the air and how those packets move through the current peripheral stack.

Exchange 1: advertising burst

One advertising burst from this peripheral is:

1. `ADV_IND` on channel 37
2. short RX window for a possible follow-up request
3. `ADV_IND` on channel 38
4. short RX window
5. `ADV_IND` on channel 39
6. short RX window

If no valid `LL_CONNECT_REQ` is accepted during those windows, the burst ends and the peripheral waits for the next advertising timer tick.

Exchange 2: `ADV_IND` followed by `LL_CONNECT_REQ`

The connection-opening packet flow is:

1. peripheral sends `ADV_IND`
2. central sends `LL_CONNECT_REQ`
3. `radio_evt_handler()` recognizes the received advertising-channel packet
4. `controller_apply_connect_request()` copies link parameters into `m_link`
5. advertising stops
6. the radio switches to connected data-channel format
7. the first connection event is scheduled

There is no extra host-side accept packet. Once the connect request is applied, the controller is in connected mode.

Exchange 3: first connected packets and empty PDUs

At the beginning of a connection, the first connected packets are often empty or control-oriented.

For every received connected packet:

1. the packet lands in `m_ctrl_rt.conn_rx_pdu`
2. `radio_handle_connected_packet()` checks `SN` to decide whether it is new
3. it checks `NESN` to decide whether the previous peripheral TX was acknowledged

4. it immediately transmits one of:
    - the last unacknowledged TX PDU again
    - a pending TX PDU prepared earlier
    - an empty PDU

This is the key timing rule of the implementation: respond first, then parse the protocol meaning of the packet.

Exchange 4: LL control PDU handling

When LLID is `11`, the payload is treated as LL control.

The path is:

1. log the control opcode into diagnostics
2. call `ll_control_process()`
3. optionally build a control response
4. store that response in the pending TX slot for a later event

Implemented control exchanges include:

- `FEATURE_REQ` or `SLV_FEATURE_REQ` -> `FEATURE_RSP`
- `VERSION_IND` -> `VERSION_IND`
- `LENGTH_REQ` -> `LENGTH_RSP` advertising 27-byte payload support
- `PHY_REQ` -> `PHY_RSP`
- `CHANNEL_MAP_IND` -> stored now, applied later at the specified instant
- `TERMINATE_IND` -> immediate disconnect handling

Exchange 5: ATT MTU exchange

When the peer sends ATT `Exchange MTU Request`:

1. the packet arrives as LLID `10`
2. the payload starts with the L2CAP header
3. the CID is checked for `0x0004`
4. `ble_gatt_server_process_request()` decodes ATT opcode `MTU_REQ`
5. the requested MTU is read
6. the effective MTU is clamped to 23
7. `MTU_RSP(23)` is built and queued

So on air, the ATT exchange is simply:

- central -> `MTU_REQ`
- peripheral -> `MTU_RSP(23)`

Exchange 6: ATT read request

For a normal read:

1. central sends `READ_REQ(handle)`
2. the controller extracts ATT from the L2CAP payload
3. `att_handle_read()` finds the target attribute
4. the response bytes are copied into `m_att_rsp`

5. the controller wraps that ATT response in L2CAP
6. the response is queued for the next transmit opportunity

On air this becomes:

- central -> `READ_REQ`
- peripheral -> `READ_RSP`

Exchange 7: ATT write request

For a characteristic write with response:

1. central sends `WRITE_REQ(handle, value...)`
2. the controller performs the immediate radio response step
3. ATT bytes are passed to `ble_gatt_server_process_request()`
4. `att_handle_write_req()` validates permissions and length
5. accepted value bytes are copied into the application-owned buffer
6. `WRITE_RSP` is built
7. a deferred GATT write event is queued
8. later `SWI1_EGU1_IRQHandler()` calls the characteristic callback

So this produces both:

- an on-air protocol response: `WRITE_RSP`
- a later software callback: `BLE_GATT_EVT_WRITE`

If the write is oversized, the stack rejects it instead of truncating it.

Exchange 8: ATT write command

For write-without-response:

1. central sends `WRITE_CMD(handle, value...)`
2. the write may still be accepted and copied into the value buffer
3. no ATT response is generated
4. a deferred GATT write callback is still queued if the write is accepted

If an oversized `WRITE_CMD` arrives, it is ignored.

Exchange 9: CCCD enable or disable write

Notification enable is just an ATT write to the CCCD handle:

1. central sends `WRITE_REQ(cccd_handle, 0x0001)` or `0x0000`
2. the CCCD bytes in characteristic runtime are updated
3. `notifications_enabled` is recomputed
4. `WRITE_RSP` is built
5. a deferred notify-enabled or notify-disabled event is queued

This cached notify state is what later allows `ble_notify_characteristic()` to be a quick check.

Exchange 10: application notification packet

When the application sends a notification:

1. the app updates the characteristic value buffer
2. the app calls `ble_notify_characteristic()`
3. `ble_gatt_server_build_notification()` builds ATT:
   - opcode `HANDLE_VALUE_NOTIFICATION`
   - value handle
   - value bytes
4. `controller_queue_att_payload()` wraps that ATT payload in the L2CAP header
5. the combined packet is stored as pending TX
6. at the next connection event the peripheral sends the LLID `10` PDU

On air, the notification packet is:

1. connected LL header
2. LL payload length
3. L2CAP length
4. L2CAP CID `0x0004`
5. ATT opcode `0x1B`
6. 2-byte value handle
7. characteristic value bytes

There is no ATT response to a notification. Reliability comes only from the normal connected-data `SN` and `NESN` acknowledgment mechanism.

Exchange 11: retransmission rule

The controller keeps:

- the most recent transmitted connected PDU
- a flag telling whether it has been acknowledged yet

So the packet-level rule is:

- if the central acknowledges the previous TX, the peripheral may move on
- if the central does not acknowledge it, the peripheral retransmits the same PDU at the next event

This is why notifications and ATT responses both use the same controller TX machinery.

Exchange 12: supervision-timeout disconnect

If valid RX packets stop arriving for long enough:

1. `TIMER2_IRQHandler()` notices repeated missed intervals
2. the accumulated miss time reaches the supervision timeout
3. `controller_disconnect_internal()` clears connected state
4. a deferred `BLE_GAP_EVT_DISCONNECTED` is queued
5. the application BLE event handler later decides whether to restart advertising

So the actual packet flow ends before the disconnect callback. The callback is just how the application is informed after link loss has already been decided.

BLE Stack Working Flow

This section describes the full runtime flow in the order the code actually executes it. If you want to understand the stack as a moving system instead of as isolated files, this is the most important section to read.

Flow 1: boot to first advertising event

1. `main()` initializes clocks, logging, LEDs, and app timers.
2. `ble_stack_init()` resets runtime state and prepares the BLE core.
3. `ble_register_evt_handler()` installs the application's unified BLE event callback.
4. `controller_load_identity_address()` creates the static random address used for advertising.
5. `ble_evt_dispatch_init()` prepares the deferred callback queue and enables `SWI1_EGU1`.
6. `controller_runtime_init()` powers and configures the radio, creates the repeated advertising timer, and registers the radio ISR callback.
7. `ble_adv_init()` copies the app's advertising config into `m_host`.
8. `ble_gap_init()` stores the preferred GAP connection parameters in `m_host`.
9. `ble_gatt_server_init()` rebuilds the full ATT database from fixed GAP/GATT attributes plus user-provided services and characteristics.
10. `ble_start_advertising()` marks the controller as advertising and starts the repeated advertising timer.
11. The repeated timer fires `adv_timer_handler()`, which calls `ble_advertise()`.
12. `ble_advertise()` builds the current advertising payload from `m_host` and transmits one advertising event across channels 37, 38, and 39.

The important design idea is that configuration lives in `m_host`, but actual on-air activity lives in `m_ctrl_rt` and is triggered by timers and interrupts.

Flow 2: one advertising event in detail

One advertising event is not one packet. In this stack it means a short burst over the three advertising channels.

The sequence is:

1. `host_build_adv_pdu()` writes a fresh `ADV_IND` packet into `m_ctrl_rt.air_pdu`.
2. The controller selects advertising channel 37 and transmits.
3. The radio is left briefly in receive mode to see whether a requester sends a valid follow-up packet such as `LL_CONNECT_REQ`.
4. If no connect request is accepted, the controller moves to channel 38 and repeats the same TX then short RX window.
5. The same happens on channel 39.
6. After all three channels are covered, the advertising event ends and the stack waits for the next advertising timer tick.

So the repeated app timer defines advertising cadence, while the per-channel loop inside `ble_advertise()` defines the structure of one advertising event.

Flow 3: connect request to connected state

When a central wants to connect, the key transition happens entirely inside the radio receive path.

1. During the RX window after an `ADV_IND`, the radio receives a packet into `m_ctrl_rt.air_pdu`.
2. `radio_evt_handler()` runs and inspects the received advertising-channel packet.
3. If the PDU type is `LL_CONNECT_REQ`, the handler calls `controller_apply_connect_request()`.
4. `controller_apply_connect_request()` parses the peer-provided connection parameters and writes the link state into `m_link`.
5. It resets connected-session runtime such as sequence numbers, retransmission state, channel selection state, supervision bookkeeping, and ATT/GATT per-connection state.
6. It stops the advertising timer because the device is no longer in the advertising state.
7. It switches the radio configuration from advertising format to connected data-channel format.
8. It schedules the first connection event using the transmit-window information from the connect request.
9. It queues a deferred `BLE_GAP_EVT_CONNECTED`.
10. Later, `SWI1_EGU1_IRQHandler()` drains the deferred event queue and calls the application's BLE event handler.

This split is very intentional: time-sensitive radio state changes happen immediately in controller code, while application notification is deferred to a safer lower-priority software interrupt.

Flow 4: one connection interval timeline

Once connected, the stack repeats a timing cycle driven by `TIMER2`.

The timeline of one interval is:

1. `TIMER2_IRQHandler()` fires at the scheduled anchor time for the next event.
2. It checks whether the connection is still valid and whether supervision timeout rules were violated.
3. It advances the next compare so the next connection event is already scheduled.
4. It clears per-event flags such as whether RX was seen in this interval and whether the first anchor packet was already captured.
5. It applies a deferred channel-map update if its instant has been reached.
6. It calls `controller_start_connection_event()`.
7. `controller_start_connection_event()` selects the next data channel, points `PACKETPTR` to `m_ctrl_rt.conn_rx_pdu`, and starts RX on that channel.
8. If the peer transmits, `radio_evt_handler()` forwards to `radio_handle_connected_packet()`.
9. `radio_handle_connected_packet()` validates the packet, updates sequencing and acknowledgment state, and immediately prepares the radio response needed for BLE timing.
10. Only after the immediate response path is handled does the stack parse the received payload for LL control or ATT meaning.
11. Any response that does not need to be sent in the current turn-around is prepared into the pending TX slot for a later event.

This explains an important reading rule for the code: on-air timing comes first, protocol interpretation comes second.

Flow 5: peer writes a characteristic

This is the most useful end-to-end data path to understand because it shows how the controller, ATT/GATT layer, and deferred callback path fit together.

1. The peer sends a data-channel PDU carrying an L2CAP payload with CID `0x0004` for ATT.
2. `radio_handle_connected_packet()` recognizes it as a new packet and performs the immediate BLE response step.
3. The controller extracts the L2CAP payload and passes the ATT bytes to `ble_gatt_server_process_request(`
4. `ble_gatt_server_process_request()` decodes the ATT opcode.
5. If it is a write, it routes to `att_handle_write_req()`.
6. `att_handle_write_req()` finds the target attribute in the built ATT table.
7. For a characteristic value write, it copies the incoming bytes directly into the application-owned value buffer referenced by that attribute.
8. If the incoming write is oversized, it is rejected instead of being truncated.
9. Otherwise it updates the stored value length.
10. It queues a deferred GATT event describing the write.
11. The software interrupt `SWI1_EGU1_IRQHandler()` later drains that event and calls the characteristic's registered callback.

That means the application callback sees the already-updated value buffer. The callback is not asked to "accept" the write; the write has already been applied when the callback runs.

Flow 6: peer enables notifications

CCCD writes use the same ATT write pipeline, but the effect is different.

1. The peer writes `0x0001` or `0x0000` to the CCCD handle of a notifying characteristic.
2. `att_handle_write_req()` recognizes that the write targets a CCCD attribute instead of a characteristic value.
3. The CCCD bytes stored in the characteristic runtime state are updated.
4. `notifications_enabled` is recomputed from the written CCCD value.
5. A deferred GATT event is queued as either `BLE_GATT_EVT_NOTIFY_ENABLED` or `BLE_GATT_EVT_NOTIFY_DISABLED`.
6. `SWI1_EGU1_IRQHandler()` later invokes the application's characteristic callback with that event.

This is why `ble_notify_characteristic()` can later make a quick decision: it only needs to check the cached notify-enabled state, not re-parse ATT state.

Flow 7: application sends a notification

The notification flow starts in normal application code rather than in the radio ISR.

1. The app updates the characteristic's backing value buffer.
2. The app calls `ble_notify_characteristic(p_char)`.
3. `ble_notify_characteristic()` validates that the link is connected and that notifications are enabled for that characteristic.
4. `ble_gatt_server_build_notification()` builds the ATT notification payload: opcode, handle, and value bytes.

5. `controller_queue_att_payload()` wraps that ATT payload in an L2CAP header and stores it in `m_ctrl_rt.pending_conn_tx_pdu`.
6. At the next suitable connection event, the controller transmits that pending PDU.
7. If the peer acknowledges it through `NESN`, the controller clears the pending state and advances sequence tracking.
8. If the peer does not acknowledge it, the controller keeps the last TX PDU and retransmits it on the next event.

This is a compact but important design: the GATT layer builds payload meaning, but only the controller owns when bytes actually go on air.

Flow 8: disconnect and return to advertising

Disconnection can come from LL terminate handling or from timeout-related link loss.

The high-level sequence is:

1. controller code decides the link must end
2. connected state in `m_link` is cleared
3. connection-timer-driven activity stops being meaningful
4. a `BLE_GAP_EVT_DISCONNECTED` event is queued
5. the application BLE event handler runs in `SWI1_EGU1_IRQHandler()`
6. the application decides what to do next, usually calling `ble_start_advertising()` again

This keeps policy in the application: the BLE core reports disconnect, but the application chooses whether to start advertising again immediately.

Diagnostics

The stack exposes a few small diagnostics helpers:

- `ble_diag_take_first_data_packet()`
- `ble_diag_take_first_ctrl_opcode()`
- `ble_diag_take_first_att_opcode()`
- `ble_diag_take_packet_trace()`

These read from `m_diag`.

What they are useful for:

- confirming first data packet arrival after connect
- seeing which LL control opcodes arrived
- seeing which ATT opcodes arrived
- tracing TX/RX packet headers and retransmissions

This is debug support, not part of the BLE protocol behavior itself.

Important Current Limitations

This section is important because it explains both the simplicity and the boundaries of the current design.

Link-layer limits

- peripheral connection acceptance is implemented

ATT/GATT limits

- ATT MTU fixed at 23
- no segmentation or reassembly
- no long reads
- no prepare write / execute write
- no indications
- no signed writes

Transmission limits

- only one pending ATT TX payload slot
- if that slot is full, notification or response queuing must wait

GAP limits

- stored preferred connection parameters are not yet actively negotiated
- no full GAP procedure set beyond minimal device name and appearance

Advertising limits

- no scan response payload
- simple `ADV_IND` advertising only

How To Read The Code Efficiently

If you want to fully understand the stack with the least confusion, read the files in this order.

Pass 1: public surface

1. `stack/include/nrf_ble.h`
2. `examples/custom_ble_stack_demo/main.c`

This tells you how the app is expected to use the stack.

Pass 2: initialization and shared state

1. `stack/core/ble_stack.c`
2. `stack/core/ble_internal.h`
3. `stack/core/ble_runtime.c`

This gives you the state containers and the event-delivery model.

Pass 3: advertising and connection engine

1. `stack/core/ble_controller.c`

Read this in chunks:

- advertising helpers
- connection request acceptance
- connection timer
- connected packet handling
- LL control handling

Pass 4: radio abstraction

1. `stack/radio/radio_driver.h`
2. `stack/radio/radio_driver.c`

Read this after the controller so the separation is clear:

- `stack/core/ble_controller.c` decides what should happen on air
- `stack/radio/radio_driver.c` performs the actual radio register operations

Pass 5: ATT/GATT processing

1. `stack/gatt/ble_gatt_server.c`

Read in this order:

- fixed attribute definitions
- runtime structs
- GATT database builders
- read/write handlers
- ATT request dispatcher

Suggested Mental Model

If you want one simple mental model for this stack, use this:

- `m_host` is configuration
- `m_link` is connection state
- `m_ctrl_rt` is packet and radio runtime
- `ble_controller.c` decides BLE packet flow and timing
- `radio_driver.c` touches the nRF radio peripheral
- `ble_gatt_server.c` turns ATT bytes into attribute operations
- `ble_runtime.c` turns internal events into callbacks

Or even shorter:

- application updates value buffers
- ATT/GATT builds protocol payloads
- controller queues and schedules them
- radio driver performs TX/RX on the peripheral
- controller parses transport and link-layer state
- ATT/GATT handles attribute-level meaning
- software interrupt delivers callbacks back to the application

Practical Trace Examples

Example 1: connect

1. app starts advertising
2. radio receives `LL_CONNECT_REQ`
3. `controller_apply_connect_request()` fills `m_link`
4. advertising stops
5. connection timer starts
6. `BLE_GAP_EVT_CONNECTED` is queued
7. `SWI1_EGU1_IRQHandler()` calls the app BLE event handler

Example 2: peer writes text characteristic

1. radio receives data-channel PDU containing ATT write
2. controller responds immediately on-air
3. controller recognizes ATT over L2CAP CID `0x0004`
4. `ble_gatt_server_process_request()` dispatches write
5. `att_handle_write_req()` updates the app buffer
6. GATT write event is queued
7. `SWI1_EGU1_IRQHandler()` calls the characteristic write callback

Example 3: app sends notification

1. app changes characteristic value
2. app calls `ble_notify_characteristic()`
3. ATT notification payload is built
4. controller stores it as pending TX
5. at the next connection event, controller sends it
6. if the peer acknowledges it, TX state advances

Final Notes

This stack is small enough that you can understand it end to end, which is one of its biggest strengths. The main thing to remember while reading it is that the implementation is intentionally narrow: it chooses a small subset of BLE behavior and keeps the timing-sensitive parts simple.

That simplicity shows up in three places:

- fixed ATT MTU of 23
- single pending connection TX slot
- deferred callback delivery instead of a main-loop event pump

If you keep those three constraints in mind, the rest of the code becomes much easier to follow.